

P1152R0

Deprecating **volatile**

Published Proposal, 2018-10-01

This version:

<http://wg21.link/P1152R0>

Issue Tracking:

[Inline In Spec](#)

Author:

[JF Bastien](#) (Apple)

Audience:

SG1, LEWG, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/D1152R0.bs

Table of Contents

- 1 **Abstract**
- 2 **A Syntax of Three Parts**
- 3 **The Wise Programmer's Fear**
 - 3.1 Overview
 - 3.2 Proposed changes
 - 3.3 When is **volatile** useful?
 - 3.4 How we got here
 - 3.4.1 Original intent for **volatile** in C
 - 3.4.2 C89 intent
 - 3.4.3 Intent in C++

- 3.5 Current Wording
- 3.6 Why the proposed changes?
 - 3.6.1 External modification
 - 3.6.2 Compound assignment
 - 3.6.3 `volatile` qualified member functions
 - 3.6.4 `volatile` overloads in the Library
 - 3.6.5 `volatile` atomic
 - 3.6.6 `volatile` parameters and returns

4 The Slow Regard of Syntactic Things

5 The Doors of Stone

6 Acknowledgements

7 Examples

References

Informative References

Issues Index

§ 1. Abstract

We propose deprecating most of `volatile`. This paper explores [§3.4 How we got here](#). See [§3.2 Proposed changes](#) for a short overview, [§3.6 Why the proposed changes?](#) for details, and [§7 Examples](#). There is currently no proposed wording: this paper tries to capture all of the required context and lets WG21 choose whether to tackle everything at once or incrementally.

The proposed deprecation preserves the useful parts of `volatile`, and removes the dubious / already broken ones. This paper aims at breaking at compile-time code which is today subtly broken at runtime or through a compiler update. The paper might also break another type of code: that which doesn't exist. This removes a significant foot-gun and removes unintuitive corner cases from the languages.

§ 2. A Syntax of Three Parts

“

C and C++ have syntax for `volatile`, and it is a syntax in three parts.

The most obvious part is the abstract machine syntax, made by loads and stores present in the original program. If there is an expression that would have touched `volatile` memory in the original source, it will generate instructions by which each byte will be touched exactly once. If there had been shared memory, a signal, even `setjmp` / `longjmp`, the `volatile` would have filled the compiler with doubt, the slowness and preciseness one expects from a compiler during external modifications. If it had been part of the memory model... but no, of course, it isn't part of the memory model. In fact there are none of these things, and so the syntax remains.

Inside C, pairs of operations can huddle with `++`, `--`, or `op=`. They're used with quiet determination, avoiding serious code. In doing this with `volatile` C adds a small, sullen syntax to the larger, hollow one. It makes an alloy of sorts, a counterpoint.

The third syntax is not an easy thing to notice. If you read the Standard for hours, you might begin to notice it in the Standard Library under its specializations and in the rough, splintered applications of design guidelines. It adds weight to Generic Programs which hold the instantiations of templates long specialized. It is in the slow back and forth of code reviews rubbing out esoteric corner cases. And it is all in C++, adding to classes that already are qualified through `const`.

The C++ Committee can move with the subtle certainty that comes from knowing many things.

`volatile` is ours, just as the third syntax is ours. This is appropriate, as it is the most onerous syntax of the three, wrapping the others inside itself. It is as deep and wide as `const`-qualification. It is heavy as a great river-smooth stone. It is the patient, cut-flower syntax of a feature which is waiting to be deprecated.

— The Name of `volatile` in the style of [\[NotW\]](#)

”

§ 3. The Wise Programmer's Fear

“

There are three things all wise programmers fear: C's corner cases, a hardware platform with no documentation, and the anger of an optimizing compiler.

— *The Name of `volatile` in the style of [NotW]*

”

§ 3.1. Overview

`volatile` is often revered as a sacred decree from C, yet very little is known about what it actually means. Further, that knowledge is often disjoint from how `volatile` is actually used. In this section we'll lay out what we want to change, explain what is useful, explain how the language got to where it is, present what the C and C++ standards say and how they got there, and finally we'll justify what should be revised.

§ 3.2. Proposed changes

This proposal has the following goals:

1. Continue supporting the time-honored usage of `volatile` to load and store variables that are used for shared memory, signal handling, `setjmp` / `longjmp`, or other external modifications such as special hardware support.
2. Deprecate (and eventually remove) `volatile` compound assignment `op=`, and pre / post increment / decrement `--` `++`.
3. Deprecate (and eventually remove) `volatile`-qualification of member functions. Don't change `volatile`-qualification of data members.
4. Deprecate (and eventually remove) partial template specializations involving `volatile`, overloads on `volatile`, and qualified member functions for all but the `atomic` and `numeric_limits` parts of the Library.
5. Deprecate (and eventually remove) `volatile` member functions of `atomic` in favor of new template partial specializations which will only declare `load`, `store`, and only exist when `is_always_lock_free` is `true`. Preserve most `volatile` free function overloads for atomic.

6. Deprecate (and eventually remove) non-reference and non-pointer `volatile` parameters. Deprecate (and eventually remove) `const` as well as `volatile` return values. References and pointers to `volatile` data remain valid.

A rationale for each of these is provided in [§3.6 Why the proposed changes?](#).

§ 3.3. When is `volatile` useful?

“

Knowing your own ignorance is the first step to enlightenment.

— *The Wise Man's Fear* [\[WMF\]](#)

”

Colloquially, `volatile` tells the compiler to back off and not optimize code too much. If the source code contains a load or store of a `volatile` operation then these should occur as many times in the final execution. A `volatile` operation cannot be eliminated or fused with a subsequent one, even if the compiler thinks that it can prove that it's useless. A `volatile` operation cannot be speculated, even if the compiler can undo or otherwise make that speculation benign.

Importantly, `volatile` does not guarantee that memory operations won't tear, meaning that a `volatile` load may observe partial writes and `volatile` stores may be observed in parts. Realistically, compilers will only tear when the hardware doesn't have an instruction which can perform the entire memory operation atomically. That being said, the Standard technically allows an implementation which touched each target byte exactly once, one after the other, in an unspecified order that could change on each execution.

The order of `volatile` operations cannot change relative to other `volatile` operations, but may change relative to non-`volatile` operations.

That being said, `volatile` doesn't imply any observable ordering in terms of the C++ memory model. Atomic instructions guarantee sequential consistency for data-race free programs (data races are otherwise undefined behavior) [\[BATTY\]](#). `volatile` has no such guarantee and doesn't imply a memory ordering or any fencing, though some implementations provide stronger guarantees (such as [\[XTENSA\]](#) and [\[MSVC\]](#)). This is not in contradiction with the previous paragraph: the *instructions* are emitted in a defined order, but processors can issue and execute them out of order, and other cores may observe them in a completely different order if no extra synchronization is used. Such synchronization can come from implementation guarantees or hardware mapping specifics.

`volatile` is nonetheless a useful concept to have at the level of the language. It is more practical than inline assembly because it lives within the language and offers fairly portable semantics for load and store. It is more capable than externally linked assembly functions (such as defined in `.S` files) because compilers don't typically inline these functions.

§ 3.4. How we got here

When discussing `volatile` in C++ it is important to understand that `volatile` came from C, before either language acquired a memory model and acknowledged the existence of threads.

§ 3.4.1. Original intent for `volatile` in C

[\[SEBOR\]](#) lays out the original intent for `volatile` in C:

The use case that motivated the introduction of the `volatile` keyword into C was a variant of the following snippet copied from early UNIX sources [\[SysIII\]](#):

```
#define KL 0177560

struct { char lobyte, hibyte; };
struct { int ks, kb, ps, pb; };

getchar() {
    register rc;
    ...
    while (KL->ks.lobyte >= 0);
    rc = KL->kb & 0177;
    ...
    return rc;
}
```

The desired effect of the while loop in the `getchar()` function is to iterate until the most significant (sign) bit of the keyboard status register mapped to an address in memory represented by the `KL` macro (the address of the memory-mapped `KBD_STAT` I/O register on the PDP-11) has become non-zero, indicating that a key has been pressed, and then return the character value extracted from the low 7 bits corresponding to the pressed key. In order for the function to behave as expected, the compiler must emit an instruction to read a value from the I/O register on each iteration of the loop. In particular, the compiler must avoid caching the read value in a CPU register and substituting it in subsequent accesses.

On the other hand, in situations where the memory location doesn't correspond to a special memory-mapped register, it's more efficient to avoid reading the value from memory if it happens to already have been read into a CPU register, and instead use the value cached in the CPU register.

The problem is that without some sort of notation (in K&R C there was none) there would be no way for a compiler to distinguish between these two cases. The following paragraph quoted from *The C Programming Language, Second Edition* [\[KR\]](#), by Kernighan and Ritchie, explains the solution that was introduced into standard C to deal with this problem: the `volatile` keyword.

The purpose of `volatile` is to force an implementation to suppress optimization that could otherwise occur. For example, for a machine with memory-mapped input/output, a pointer to a device register might be declared as a pointer to `volatile`, in order to prevent the compiler from removing apparently redundant references through the pointer.

Using the `volatile` keyword, it should then be possible to rewrite the loop in the snippet above as follows:

```
while (*(volatile int*)&KL->ks.lobyte >= 0);
```

or equivalently:

```
volatile int *lobyte = &KL->ks.lobyte;  
while (*lobyte >= 0);
```

and prevent the compiler from caching the value of the keyboard status register, thus guaranteeing that the register will be read once in each iteration.

The difference between the two forms of the rewritten loop is of historical interest: Early C compilers are said to have recognized the first pattern (without the `volatile` keyword) where the address used to access the register was a constant, and avoided the undesirable optimization for such accesses [GWYN]. However, they did not have the same ability when the access was through pointer variable in which the address had been stored, especially not when the use of such a variable was far removed from the last assignment to it. The `volatile` keyword was intended to allow both forms of the loop to work as expected.

The use case exemplified by the loop above has since become idiomatic and is being extensively relied on in today's software even beyond reading I/O registers.

As a representative example, consider the Linux kernel which relies on `volatile` in its implementation of synchronization primitives such as spin locks, or for performance counters. The variables that are operated on by these primitives are typically declared to be of unqualified (i.e., non `volatile`) scalar types and allocated in ordinary memory. In serial code, for maximum efficiency, each such variable is read and written just like any other variable, with its value cached in a CPU register as compiler optimizations permit. At well-defined points in the code where such a variable may be accessed by more than one CPU at a time, the caching must be prevented and the variable must be accessed using the special `volatile` semantics. To achieve that, the kernel defines two macros: `READ_ONCE`, and `WRITE_ONCE`, in whose terms the primitives are implemented. Each of the macros prevents the compiler optimization by casting the address of its argument to a `volatile T*` and accessing the variable via an lvalue of the `volatile`-qualified type `T` (where `T` is one of the standard scalar types). Other primitives guarantee memory synchronization and visibility but those are orthogonal to the subject of this paper. See [P0124R5].

Similar examples can be found in other system or embedded programs as well as in many other pre-C11 and pre-C++11 code bases that don't rely on the Atomic types and operations newly introduced in those standards. . They are often cited in programming books [\[CBOOK\]](#) and in online articles [\[INTRO\]](#) [\[WHY\]](#) [\[WHYC\]](#).

§ 3.4.2. C89 intent

[\[RATIONALE\]](#) lays out the intent for `volatile` in C89:

The C89 Committee concluded that about the only thing a strictly conforming program can do in a signal handler is to assign a value to a volatile static variable which can be written uninterruptedly and promptly return.

[...]

volatile: No cacheing through this lvalue: each operation in the abstract semantics must be performed (that is, no cacheing assumptions may be made, since the location is not guaranteed to contain any previous value). In the absence of this qualifier, the contents of the designated location may be assumed to be unchanged except for possible aliasing.

[...]

A **static volatile** object is an appropriate model for a memory-mapped I/O register. Implementors of C translators should take into account relevant hardware details on the target systems when implementing accesses to **volatile** objects. For instance, the hardware logic of a system may require that a two-byte memory-mapped register not be accessed with byte operations; and a compiler for such a system would have to assure that no such instructions were generated, even if the source code only accesses one byte of the register. Whether read-modify-write instructions can be used on such device registers must also be considered. Whatever decisions are adopted on such issues must be documented, as **volatile** access is implementation-defined. A **volatile** object is also an appropriate model for a variable shared among multiple processes.

A **static const volatile** object appropriately models a memory-mapped input port, such as a real-time clock. Similarly, a **const volatile** object models a variable which can be altered by another process but not by this one.

[...]

A cast of a value to a qualified type has no effect; the qualification (**volatile**, say) can have no effect on the access since it has occurred prior to the cast. If it is necessary to access a non-**volatile** object using **volatile** semantics, the technique is to cast the address of the object to the appropriate pointer-to-qualified type, then dereference that pointer.

[...]

The C89 Committee also considered requiring that a call to **longjmp** restore the calling environment fully, that is, that upon execution of **longjmp**, all local variables in the environment of **setjmp** have the values they did at the time of the **longjmp** call. Register variables create problems with this idea. Unfortunately, the best that many implementations attempt with register variables is to save them in **jmp_buf** at the time of the initial **setjmp** call, then restore them to that state on each return initiated by a **longjmp** call. Since compilers are certainly at liberty to

change register variables to automatic, it is not obvious that a register declaration will indeed be rolled back. And since compilers are at liberty to change automatic variables to register if their addresses are never taken, it is not obvious that an automatic declaration will not be rolled back, hence the vague wording. In fact, the only reliable way to ensure that a local variable retain the value it had at the time of the call to `longjmp` is to define it with the `volatile` attribute.

§ 3.4.3. Intent in C++

`volatile` was extended to "fit" into C++ by allowing `volatile`-qualification to member functions. [\[DE\]](#) states:

To match ANSI C, the `volatile` modifier was introduced to help optimizer implementers. I am not at all sure that the syntactic parallel with `const` is warranted by semantic similarities. However, I never had strong feelings about `volatile` and see no reason to try to improve on the ANSI C committee's decisions in this area.

As threads and a formal model were added to C++ it was unclear what role `volatile` should play. It was often advocated for multi-threaded applications [\[INTRO\]](#). This advice is incorrect as [\[ROBISON\]](#) explains. Others, such as [\[ALEXANDRESCU\]](#), suggested using `volatile` member functions to have the type system enforce user annotations about thread safety. Various approaches were suggested to improve visibility of writes in a memory model—such as [\[REGEHR\]](#)—but weren't adopted for C++. [\[N2016\]](#) explains why `volatile` shouldn't acquire atomicity and thread visibility semantics. Further, [\[BOEHM\]](#) makes the a case that threads cannot be implemented as a library. A variety of FAQs existed to help programmers make sense of the state of concurrency before C++0x became C++11, for example [\[FAQ\]](#). The behavior of `volatile` has slightly changed over time [\[CWG1054\]](#) [\[CHANGE\]](#) [\[WHEN\]](#). Importantly, C++11's memory model forbids the compiler from introducing races in otherwise correct code, modulo compiler bugs [\[INVALID\]](#).

§ 3.5. Current Wording

“

C++ has flaws, but what does that matter when it comes to matters of the heart? We love what we love. Reason does not enter into it. In many ways, unwise love is the truest love. Anyone can love a thing *because*. That’s as easy as putting a penny in your pocket. But to love something *despite*. To know the flaws and love them too. That is rare and pure and perfect.

— *The Wise Programmer’s Fear* in the style of [\[WMF\]](#)

”

The above description doesn’t tell us *how* **volatile** is used: it merely sets out, informally, what it guarantees and what the intent was. What follows are the formal guarantees provided by **volatile**. As of this writing, the word **volatile** appears 322 times in the current draft of the C++ Standard [\[DRAFT\]](#). Here are the salient appearances from C++17 [\[N4659\]](#):

Program execution [**intro.execution**]

Accesses through **volatile** glvalues are evaluated strictly according to the rules of the abstract machine.

Reading an object designated by a **volatile** glvalue, modifying an object, calling a library I/O function, or calling a function that does any of those operations are all *side effects*, which are changes in the state of the execution environment. *Evaluation* of an expression (or a subexpression) in general includes both value computations (including determining the identity of an object for glvalue evaluation and fetching a value previously assigned to an object for prvalue evaluation) and initiation of side effects. When a call to a library I/O function returns or an access through a **volatile** glvalue is evaluated the side effect is considered complete, even though some external actions implied by the call (such as the I/O itself) or by the **volatile** access may not have completed yet.

Data races [**intro.races**]

Two accesses to the same object of type `volatile std::sig_atomic_t` do not result in a data race if both occur in the same thread, even if one or more occurs in a signal handler. For each signal handler invocation, evaluations performed by the thread invoking a signal handler can be divided into two groups A and B, such that no evaluations in B happen before evaluations in A, and the evaluations of such `volatile std::sig_atomic_t` objects take values as though all evaluations in A happened before the execution of the signal handler and the execution of the signal handler happened before all evaluations in B.

Forward progress [**intro.progress**]

The implementation may assume that any thread will eventually do one of the following:

- terminate,
- make a call to a library I/O function,
- perform an access through a `volatile` glvalue, or
- perform a synchronization operation or an atomic operation

During the execution of a thread of execution, each of the following is termed an execution step:

- termination of the thread of execution,
- performing an access through a `volatile` glvalue, or
- completion of a call to a library I/O function, a synchronization operation, or an atomic operation.

Class member access [**expr.ref**]

Abbreviating *postfix-expression.id-expression* as `E1.E2`, `E1` is called the *object expression*. If `E2` is a bit-field, `E1.E2` is a bit-field. The type and value category of `E1.E2` are determined as follows. In the remainder of [expr.ref], *cq* represents either `const` or the absence of `const` and *vq* represents either `volatile` or the absence of `volatile`. *cv* represents an arbitrary set of cv-qualifiers.

- If `E2` is a non-static data member and the type of `E1` is “*cq1 vq1 X*”, and the type of `E2` is “*cq2 vq2 T*”, the expression designates the named member of the object designated by the first expression. If `E1` is an lvalue, then `E1.E2` is an lvalue; otherwise `E1.E2` is an xvalue. Let the notation *vq12* stand for the “union” of *vq1* and *vq2*; that is, if *vq1* or *vq2* is `volatile`, then *vq12* is `volatile`. Similarly, let the notation *cq12* stand for the “union” of *cq1* and *cq2*; that is, if *cq1* or *cq2* is `const`, then *cq12* is `const`. If `E2` is declared to be a `mutable` member, then the type of `E1.E2` is “*vq12 T*”. If `E2` is not declared to be a `mutable` member, then the type of `E1.E2` is “*cq12 vq12 T*”.

The cv-qualifiers [**dcl.type.cv**]

The semantics of an access through a `volatile` glvalue are implementation-defined. If an attempt is made to access an object defined with a `volatile`-qualified type through the use of a non-`volatile` glvalue, the behavior is undefined.

[*Note*: `volatile` is a hint to the implementation to avoid aggressive optimization involving the object because the value of the object might be changed by means undetectable by an implementation. Furthermore, for some implementations, `volatile` might indicate that special hardware instructions are required to access the object. See [intro.execution] for detailed semantics. In general, the semantics of `volatile` are intended to be the same in C++ as they are in C. —end note]

Non-static member functions [**class.mfct.non-static**]

A non-static member function may be declared `const`, `volatile`, or `const volatile`. These cv-qualifiers affect the type of the `this` pointer. They also affect the function type of the member function; a member function declared `const` is a `const` member function, a member function declared `volatile` is a `volatile` member function and a member function declared `const volatile` is a `const volatile` member function.

The `this` pointer [**class.this**]

In the body of a non-static member function, the keyword `this` is a prvalue expression whose value is the address of the object for which the function is called. The type of `this` in a member function of a class `X` is `X*`. If the member function is declared `const`, the type of `this` is `const X*`, if the member function is declared `volatile`, the type of `this` is `volatile X*`, and if the member function is declared `const volatile`, the type of `this` is `const volatile X*`.

`volatile` semantics apply in `volatile` member functions when accessing the object and its non-static data members.

Constructors [`class.ctor`]

A constructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics are not applied on an object under construction. They come into effect when the constructor for the most derived object ends.

Destructors [`class.dtor`]

A destructor is used to destroy objects of its class type. The address of a destructor shall not be taken. A destructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics are not applied on an object under destruction. They stop being in effect when the destructor for the most derived object starts.

Overloadable declarations [`over.load`]

Parameter declarations that differ only in the presence or absence of `const` and/or `volatile` are equivalent. That is, the `const` and `volatile` type-specifiers for each parameter type are ignored when determining which function is being declared, defined, or called.

Built-in operators [`over.built`]

In the remainder of this section, *vq* represents either `volatile` or no cv-qualifier.

For every pair (*T*, *vq*), where *T* is an arithmetic type other than `bool`, there exist candidate operator functions of the form

```
vq T & operator++(vq T &);  
T operator++(vq T &, int);
```

For every pair (*T*, *vq*), where *T* is an arithmetic type other than `bool`, there exist candidate operator functions of the form

```
vq T & operator--(vq T &);  
T operator--(vq T &, int);
```

For every pair (*T*, *vq*), where *T* is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```
T*vq& operator++(T*vq&);  
T*vq& operator--(T*vq&);  
T* operator++(T*vq&, int);  
T* operator--(T*vq&, int);
```

For every quintuple (*C1*, *C2*, *T*, *cv1*, *cv2*), where *C2* is a class type, *C1* is the same type as *C2* or is a derived class of *C2*, and *T* is an object type or a function type, there exist candidate operator functions of the form

```
cv12 T& operator->*(cv1 C1*, cv2 T C2::*);
```

For every triple (*L*, *vq*, *R*), where *L* is an arithmetic type, and *R* is a promoted arithmetic type, there exist candidate operator functions of the form

```
vq L& operator=(vq L&, R);  
vq L& operator*=(vq L&, R);  
vq L& operator/=(vq L&, R);  
vq L& operator+=(vq L&, R);  
vq L& operator-=(vq L&, R);
```

For every pair (*T*, *vq*), where *T* is any type, there exist candidate operator functions of the form


```
T*vq& operator=(T*vq&, T*);
```

For every pair (T, vq) , where T is an enumeration or pointer to member type, there exist candidate operator functions of the form

```
vq T& operator=(vq T&, T );
```

For every pair (T, vq) , where T is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```
T*vq& operator+=(T*vq&, std::ptrdiff_t);  
T*vq& operator-=(T*vq&, std::ptrdiff_t);
```

For every triple (L, vq, R) , where L is an integral type, and R is a promoted integral type, there exist candidate operator functions of the form

```
vq L& operator%=(vq, L&, R);  
vq L& operator<=<=(vq, L&, R);  
vq L& operator>=>=(vq, L&, R);  
vq L& operator&=(vq, L&, R);  
vq L& operator^=(vq, L&, R);  
vq L& operator|=(vq, L&, R);
```

Here are salient appearances of `volatile` in the C17 Standard:

Type qualifiers

An object that has `volatile`-qualified type may be modified in ways unknown to the implementation or have other unknown side effects. Therefore any expression referring to such an object shall be evaluated strictly according to the rules of the abstract machine. Furthermore, at every sequence point the value last stored in the object shall agree with that prescribed by the abstract machine, except as modified by the unknown factors mentioned previously.[†] What constitutes an access to an object that has `volatile`-qualified type is implementation-defined.

[†] A `volatile` declaration may be used to describe an object corresponding to a memory-mapped input/output port or an object accessed by an asynchronously interrupting function. Actions on objects so declared shall not be "optimized out" by an implementation or reordered except as permitted by the rules for evaluating expressions.

§ 3.6. Why the proposed changes?

“

Only priests and fools are fearless and I’ve never been on the best of terms with God.

— *The Name of The Wind* [\[NotW\]](#)

”

§ 3.6.1. External modification

We’ve shown that `volatile` is purposely defined to denote external modifications. This happens for:

- Shared memory with untrusted code, where `volatile` is the right way to avoid time-of-check time-of-use (ToCToU) races which lead to security bugs such as [\[PWN2OWN\]](#) and [\[XENXSA155\]](#).
- Signal handling, where at any time in a program’s execution a signal can occur, and the optimizer must therefore make sure that `volatile` operations occur (even though they are allowed to tear).
- `setjmp` / `longjmp`, where `setjmp` can effectively return twice and `volatile` is used to prevent motion of memory operations around this operation. `atomic_signal_fence` is a more coarse-grained solution to this problem.
- Various other external modifications such as special hardware support—e.g. memory-mapped registers—where the compiler cannot assume that memory doesn’t change or that writes aren’t synchronizing externally.

- Marking that an infinite loop has side-effects and is therefore not undefined behavior (this can also be done with atomic or I/O operations).
- Casting pointers to `volatile` to denote code expecting `volatile` semantics (as opposed having this as a property of data). This type of code is commonplace and we intend this paper to leave it alone, e.g. [\[TORVALDS\]](#).
- Enforcing control dependencies and preventing compiler value speculation (such as through feedback-directed optimization) as discussed in [\[CONTROL\]](#).
- Avoiding value speculation around some hand-rolled implementations of `memory_order_consume` (until [\[P0750R1\]](#) is resolved).

As [\[SEBOR\]](#) lays out there have been wording issues around this usage. [\[TROUBLE\]](#) and [\[ACCESS_ONCE\]](#) make a similar case. This paper doesn't try to address those issues. We don't see a reason to change existing syntax denoting external modification in this paper: this paper rather focuses on deprecation of *invalid* or *misleading* uses of `volatile`. The above uses are valid and have no alternative other than inline assembly.

`volatile` deprecation / repurposing in any form wasn't on the table when C++11 was standardized because existing code had no alternative but (sometimes erroneous) `volatile` coupled with inline assembly. Now that codebases use atomic and have moved away from *erroneous* `volatile`, we believe deprecation is warranted. In other words, what would have been a disastrous breaking change for C++11 is merely good house-cleaning for C++20.

A new language would likely do things differently, but this paper isn't about creating a new language. Notably, [\[D\]](#) and [\[Rust\]](#) took different approaches (`peek/poke` and `unsafe read_volatile<T>` / `unsafe write_volatile<T>` respectively).

Note that an important aspect of external modification for `volatile` is `constexpr`. As discussed in [\[CWG1688\]](#) a `constexpr volatile` is intentionally permitted and could be used in some circumstances to force constant initialization.

§ 3.6.2. Compound assignment

`volatile` external modifications are only truly meaningful for loads and stores. Other read-modify-write operations imply touching the `volatile` object more than once per byte because that's fundamentally how hardware works. Even atomic instructions (remember: `volatile` isn't atomic) need to read *and* write a memory location (e.g. x86's `lock addl $42, (%rdi)` won't allow a race between the read and write, but needs to both read and write, whereas ARM will require a load-linked store-conditional loop to perform the same operation). These RMW operations are therefore

misleading and should be spelled out as separate `read` ; `modify` ; `write`, or use `volatile` atomic operations which we discuss below.

We propose to deprecate, and eventually remove, `volatile` compound assignment `op=`, and pre / post increment / decrement `--` `++` of `volatile` variables. This is a departure from C which breaks source compatibility (once removed), but maintains ABI compatibility.

ISSUE 1 We would like guidance on `volatile` when combined with `operator->*`. That guidance will depend on choices made with respect to `volatile` aggregates.

ISSUE 2 There's a related problem in [\[TROUBLE\]](#) with chained assignments of `volatile` values, such as `a = b = c`. This is equally misleading, and it's not intuitive whether the value stored to `b` is re-read before storing to `a`. We would like guidance on whether this is worth addressing.

§ 3.6.3. `volatile` qualified member functions

`volatile`-qualification of member functions was added to C++ to parallel `const`-qualification. Unlike `const`-qualification this never truly got used except for odd cases such as [\[ALEXANDRESCU\]](#). [\[DE\]](#) is clearly uncertain about whether `volatile`-qualification of member functions is warranted. This mis-feature is either a heavy burden on Generic Programming, or something Generic Programming purposely avoids supporting because it often doubles the (already chatty) amount of Generic code.

Let's consider what `const`-qualification truly means: a `class` could be used in a context where it can be mutated, as well as in a context where it cannot be mutated. A member function can be declared `const` to behave differently, and this qualifier can be used to forbid usage of non-`const` member functions when a variable is `const`. This doesn't translate to `volatile`: why would a `class` *sometimes* map to hardware and sometimes not? And more importantly, *how* would a member function meaningfully differ in those circumstances?

It's worth noting that `const` constructors aren't a thing: `const` semantics come into effect when the constructor for the most derived object ends. The same applies to `volatile`, but if the object was truly hardware-mapped or potentially externally modified then it seems unwise to construct its members without `volatile` semantics. Ditto for destructors.

We propose to deprecate, and eventually remove, `volatile`-qualified member functions.

Our goal is to avoid the ambiguity where an aggregate is sometimes `volatile` and sometimes not. The above proposal forces developers to recursively `volatile`-qualify all non-aggregate data members. Alternatively, we could:

1. Mandate that member functions `volatile`-qualification be all-or-nothing; or
2. Allow the aggregate declaration itself to be `volatile` (e.g. `struct volatile my_hardware { /* ... */ };`).
3. Disallow `volatile`-qualified aggregates entirely.

Either of the first two approaches clearly tell the compiler that every data member access should be done in a `volatile` manner, and disallows accessing that particular aggregate in a non-`volatile` manner. In all cases, data members can still be `volatile`-qualified.

ISSUE 3 Which of the above approaches (deprecate `volatile`-qualified member functions, all-or-nothing, `struct volatile`, or deprecate `volatile` aggregates, should we pursue?

ISSUE 4 If we keep `volatile` aggregates, it seems like `volatile` aggregates should have constructors which initialize all members with `volatile` semantics, and destroy them with `volatile` semantics. Otherwise, we encourage the use of initialization / cleanup member functions. Alternatively, triviality could be mandated for constructors and destructors of `volatile` aggregates. The author is told by some embedded developers that it's very common to have an aggregate that describes some hardware, and to access hardware registers (member variables of the aggregate) by indirection through a pointer to `volatile` struct.

ISSUE 5 If we keep `volatile` aggregates, what does it mean to have a `volatile` virtual function table pointer?

ISSUE 6 It is unclear how `volatile union` should be accessed when all types in the `union` aren't stored using the same bits (i.e. should `union { char c; int i; }` always access the full union, even when only accessing `c`?). Some hardware defines different semantics to MMIO register accesses of different sizes to the same address. A union would be a natural way to represent such hardware. This could be outside the scope of the current paper.

ISSUE 7 We would like guidance on whether `volatile` bit-fields should be constrained. At the moment no guarantee is made about read-modify-write operations required when mixing `volatile` and non-`volatile` bit-field data members. It seems like, at a minimum, bit fields should always cause alignment when subsequent data members change their cv-qualification. This could be outside the scope of the current paper.

§ 3.6.4. `volatile` overloads in the Library

Partial template specializations involving `volatile`, overloads on `volatile`, and qualified member functions are provided for the following classes:

- `numeric_limits`
- `tuple_size`
- `tuple_element`
- `variant_size`
- `variant_alternative`
- atomic free function overloads
- `atomic` member functions

`numeric_limits` is obviously useful and should stay. Atomic is discussed below. Tuple and variant are odd in how they're made to support `volatile`, and we wonder why other parts of the Library aren't consistent. It's unclear what hardware mapping is expected from a tuple, and how a `volatile` discriminated `union` (such as variant) should be accessed.

We propose to deprecate, and eventually remove, `volatile` partial template specializations, overloads, or qualified member functions for all but the `atomic` and `numeric_limits` parts of the Library.

As of this writing, `volatile` appears in libc++ as follows:

Directory	<code>volatile</code> count
<code>include/__functional_base</code>	12
<code>include/__tuple</code>	12
<code>include/atomic</code>	175
<code>include/chrono</code>	2
<code>include/limits</code>	48
<code>include/memory</code>	2

<code>include/mutex</code>	1
<code>include/scoped_allocator</code>	1
<code>include/type_traits</code>	71
<code>include/variant</code>	8
<code>src/mutex.cpp</code>	1

ISSUE 8 Should we go further and forbid `volatile` in containers? Some containers seem useful for signal handlers and such, however how loads and stores are performed to these containers isn't mandated by the Standard which means that to use containers in signal handlers one needs to synchronize separately with `atomic_signal_fence` and some form of token. Containers of `volatile` data are therefore misleading at best.

§ 3.6.5. `volatile` `atomic`

`volatile` can tear, provides no ordering guarantees (with respect to non-`volatile` memory operations, and when it comes to CPU reordering), can touch bytes exactly once, and inhibits optimizations. This is useful. `atomic` cannot tear, has a full memory model, can require a loop to succeed, and can be optimized [N4455]. This is also useful. `volatile` `atomic` should offer the union of these properties, but currently fails to do so:

- A non-lock-free `atomic` can be `volatile`, in which case it can tear when the issued instructions are considered.
- Read-modify-write operations are implemented as either loops which retry, locked instructions (which still perform a load and a store), as transactional memory operations, or as memory controller operations. Only the last of these can truly be said to touch each byte exactly once, and these hardware implementations are far from the norm.

We propose to deprecate, and eventually remove, `volatile` member functions of `atomic` in favor of new template partial specializations which will only declare `load`, `store`, and only exist when `is_always_lock_free` is `true`.

ISSUE 9 We would like guidance on whether other read-modify-write operations should be maintained with implementation-defined semantics. Specifically, `exchange`, `compare_exchange_strong`, `compare_exchange_weak`, `fetch_add` / `fetch_sub` (for integral, pointer, and floating-point), and `fetch_and` / `fetch_or` / `fetch_xor` (for integral), can be given useful semantics by an implementation which wishes to guarantee that particular instructions will be emitted. This would maintain the status-quo whereby `volatile` is a semi-portable abstraction for hardware, and still allows us to consider deprecation in the future. Keeping these for now is the conservative option.

The same guidance would apply to atomic free function overloads.

§ 3.6.6. `volatile` parameters and returns

Marking parameters as `volatile` makes sense to denote external modification through signals or `setjmp` / `longjmp`. In that sense it's similar to `const`-qualified parameters: it has clear semantics *within* the function's implementation. However, it leaks function implementation information to the caller. It also has no semantics when it comes to calling convention because it is explicitly ignored (and must therefore have the same semantics as a non-`volatile` declaration). It's much simpler to have the desirable behavior above by copying a non-`volatile` parameter to an automatic stack variable marked `volatile`. A compiler could, if stack passing is required by the ABI, make no copy at all in this case.

`volatile` return values are pure nonsense. Is register return disallowed? What does it mean for return value optimization? A caller is better off declaring a `volatile` automatic stack variable and assigning the function return to it, and the caller will be none the wiser.

Similarly, `const` return values are actively harmful. Both cv-qualifiers already have no effect when returning non-class types, and `const`-qualified class return types are harmful because they inhibit move semantics.

We propose to deprecate, and eventually remove, non-reference and non-pointer `volatile` parameters and return values. That is, `volatile` at the outermost level of the parameter type specification. We also propose to deprecate, and eventually remove, `const`-qualified return types.

§ 4. The Slow Regard of Syntactic Things

“

This paper is for all the slightly broken features out there. `volatile` is one of you. You are not alone. You are all beautiful to me.

— *The Slow Regard of Syntactic Things* in the style of [\[SRST\]](#)

”

This proposal tries to balance real-world usage, real-world breakage, frequent gotchas, and overly chatty features which aren’t actually used. The author thinks it strikes the right balance, but may be wrong. `volatile` may be better suited with slower deprecation. `volatile` might be one for direct removal instead of deprecation. `volatile` might prefer causing diagnostics. `volatile` could even consider full deprecation and replacement with `volatile_load<T>` / `volatile_store<T>` free functions. `volatile` might not suit aggregate types, maybe it should only be allowed on scalars.

It is important that `volatile` dare be entirely itself, be wild enough to change itself while somehow staying altogether *true*, lest we end up with The Silent Regard of Slow Things. Annex C should be updated, WG14 should be consulted.

§ 5. The Doors of Stone

This section will hold future work, wording, etc. It will be published based on Committee feedback, when it is ready. The author wants to give the Committee a perfectly worded paper. They deserve it.

Here are items which could be discussed in the future:

1. `asm` statements are implementation-defined. Many compilers also support `asm volatile` statements, which are also implementation-defined and not in scope for this paper.
2. Standardize a library-like replacement for `volatile` load / store, such as `peek` / `poke`.

§ 6. Acknowledgements

Early drafts were reviewed by the C++ Committee’s Direction Group, Thomas Rodgers, Arthur O’Dwyer, John McCall, Mike Smith, John Regehr, Herb Sutter, Shafik Yaghmour, Hans Boehm, Richard Smith, Will Deacon, Paul McKenney. Thank you for in-depth feedback, and apologies if I mistakenly transcribed your feedback.

Patrick Rothfuss, for writing amazing books. May he take as much time as needed to ensure forward progress of book 3.

§ 7. Examples

Here are dubious uses of `volatile`.

```
struct foo {
    int a : 4;
    int b : 2;
};
volatile foo f;
// Which instructions get generated? Does this touch the bytes more than once?
f.a = 3;
```

```
struct foo {
    volatile int a : 4;
    int b : 2;
};
foo f;
f.b = 1; // Can this touch a?
```

```
union foo {
    char c;
    int i;
};
volatile foo f;
// Must this touch sizeof(int) bytes? Or just sizeof(char) bytes?
f.c = 42;
```

```
volatile int i;
// Can each of these touch the bytes only once?
i += 42;
++i;
```

```
volatile int i, j, k;
// Does this reload j before storing to i?
i = j = k;
```

```
struct big { int arr[32]; };
volatile __Atomic struct big ba;
struct big b2;
// Can this tear?
ba = b2;
```

```
int what(volatile std::atomic<int> *atom) {
    int expected = 42;
    // Can this touch the bytes more than once?
    atom->compare_exchange_strong(expected, 0xdead);
    return expected;
}
```

```
void what_does_the_caller_care(volatile int);
```

```
volatile int nonsense(void);
```

```
struct retme { int i, j; };
volatile struct retme silly(void);
```

```
struct device {
    unsigned reg;
    device() : reg(0xc0ffee) {}
    ~device() { reg = 0xdeadbeef; }
};
volatile device dev; // Initialization and destruction aren't volatile.
```

§ References

§ Informative References

[ACCESS_ONCE]

corbet. [ACCESS_ONCE\(\)](https://lwn.net/Articles/508991/). 2012-08-01. URL: <https://lwn.net/Articles/508991/>

[ALEXANDRESCU]

Andrei Alexandrescu. [volatile: The Multithreaded Programmer's Best Friend](http://www.drdobbs.com/cpp/volatile-the-multithreaded-programmers-b/184403766). 2001-02-01. URL: <http://www.drdobbs.com/cpp/volatile-the-multithreaded-programmers-b/184403766>

[BATTY]

Mark Batty; et al. [Clarifying and Compiling C/C++ Concurrency: from C++11 to POWER](https://www.cl.cam.ac.uk/~pes20/cpppppc/popl079-batty.pdf). 2012-01-25. URL: <https://www.cl.cam.ac.uk/~pes20/cpppppc/popl079-batty.pdf>

[BOEHM]

Hans Boehm. [Threads Cannot be Implemented as a Library](http://www.hpl.hp.com/techreports/2004/HPL-2004-209.pdf). 2004-11-12. URL: <http://www.hpl.hp.com/techreports/2004/HPL-2004-209.pdf>

[CBOOK]

Mike Banahan; Declan Brady. [The C Book](http://publications.gbdirect.co.uk/c_book/chapter8/const_and_volatile.html). URL: http://publications.gbdirect.co.uk/c_book/chapter8/const_and_volatile.html

[CHANGE]

[Correct behaviour of trivial statements involving expressions with volatile variables?](https://stackoverflow.com/questions/20242868/correct-behaviour-of-trivial-statements-involving-expressions-with-volatile-variables?). 2013-11-27. URL: <https://stackoverflow.com/questions/20242868/correct-behaviour-of-trivial-statements-involving-expressions-with-volatile-variables?>

[CONTROL]

David Howells; et al. [Linux kernel memory barriers: control dependencies](https://github.com/torvalds/linux/blob/7876320f88802b22d4e2daf7eb027dd14175a0f8/Documentation/memory-barriers.txt#L666). 2018-07-17. URL: <https://github.com/torvalds/linux/blob/7876320f88802b22d4e2daf7eb027dd14175a0f8/Documentation/memory-barriers.txt#L666>

[CWG1054]

Hans Boehm. [Lvalue-to-rvalue conversions in expression statements](https://wg21.link/cwg1054). 16 March 2010. C++11. URL: <https://wg21.link/cwg1054>

[CWG1688]

Daniel Krügler. [Volatile constexpr variables](https://wg21.link/cwg1688). 18 May 2013. NAD. URL: <https://wg21.link/cwg1688>

[D]

Walter Bright. [DLang issue #13138: add peek/poke as compiler intrinsics](https://issues.dlang.org/show_bug.cgi?id=13138). 2014-07-16. URL: https://issues.dlang.org/show_bug.cgi?id=13138

[DE]

Bjarne Stroustrup. *The Design and Evolution of C++*. 1994.

[DRAFT]

[C++ Standards draft: volatile](https://github.com/cplusplus/draft/search?l=TeX&q=volatile). URL: <https://github.com/cplusplus/draft/search?l=TeX&q=volatile>

[FAQ]

Hans Boehm; Paul McKenney. [Programming with Threads: Questions Frequently Asked by C and C++ Programmers](http://www.hboehm.info/c++mm/user-faq.html). URL: <http://www.hboehm.info/c++mm/user-faq.html>

[GWYN]

[A question on volatile accesses](https://groups.google.com/forum/#!msg/comp.std.c/tHvQhiKFtD4/zfIgJhbkCXcJ). URL: <https://groups.google.com/forum/#!msg/comp.std.c/tHvQhiKFtD4/zfIgJhbkCXcJ>

[INTRO]

Nigel Jones. [Introduction to the volatile keyword](https://www.embedded.com/electronics-blogs/beginner-s-corner/4023801/Introduction-to-the-Volatile-Keyword). 2001-07-02. URL: <https://www.embedded.com/electronics-blogs/beginner-s-corner/4023801/Introduction-to-the-Volatile-Keyword>

[INVALID]

Viktor Vafeiadis; et al. [Common compiler optimisations are invalid in the C11 memory model and what we can do about it](https://people.mpi-sws.org/~viktor/papers/popl2015-c11comp.pdf). 2015/05-11. URL: <https://people.mpi-sws.org/~viktor/papers/popl2015-c11comp.pdf>

[KR]

Dennis Ritchie; Brian Kernighan. *The C Programming Language*. 1978.

[MSVC]

[Visual Studio: C++ Language Reference > Basic Concepts > Declarations and Definitions > volatile \(C++\)](https://msdn.microsoft.com/en-us/library/12a04hfd.aspx). URL: <https://msdn.microsoft.com/en-us/library/12a04hfd.aspx>

[N2016]

H. Boehm, N. Maclaren. [Should volatile Acquire Atomicity and Thread Visibility Semantics?](https://wg21.link/n2016). 21 April 2006. URL: <https://wg21.link/n2016>

[N4455]

JF Bastien. [No Sane Compiler Would Optimize Atomics](https://wg21.link/n4455). 10 April 2015. URL: <https://wg21.link/n4455>

[N4659]

Richard Smith. [Working Draft, Standard for Programming Language C++ Note.](https://wg21.link/n4659) 21 March 2017. URL: <https://wg21.link/n4659>

[NotW]

Patrick Rothfuss. [The Name of the Wind](http://www.penguin.com/ajax/books/excerpt/9780756405892). 2007-03-27. URL: <http://www.penguin.com/ajax/books/excerpt/9780756405892>

[P0124R5]

Paul E. McKenney, Ulrich Weigand, Andrea Parri, Boqun Feng. [Linux-Kernel Memory Model](https://wg21.link/p0124r5). 6 April 2018. URL: <https://wg21.link/p0124r5>

[P0750R1]

JF Bastien, Paul E. McKenney. [Consume](https://wg21.link/p0750r1). 11 February 2018. URL: <https://wg21.link/p0750r1>

[PWN2OWN]

lokihardt. [Chromium pwn2own GPU bug](https://bugs.chromium.org/p/chromium/issues/detail?id=468936). 2015-03-19. URL: <https://bugs.chromium.org/p/chromium/issues/detail?id=468936>

[RATIONALE]

[Rationale for International Standard—Programming Languages—C](http://www.open-std.org/jtc1/sc22/wg14/www/C99RationaleV5.10.pdf). 2003-04. URL: <http://www.open-std.org/jtc1/sc22/wg14/www/C99RationaleV5.10.pdf>

[REGEHR]

John Regehr; Nathan Coopridge; David Gay. [Atomicity and Visibility in Tiny Embedded Systems](https://www.cs.utah.edu/%7Eregehr/papers/plos06b.pdf). 2006-10-22. URL: <https://www.cs.utah.edu/%7Eregehr/papers/plos06b.pdf>

[ROBISON]

Arch Robison. [Volatile: Almost Useless for Multi-Threaded Programming](https://software.intel.com/en-us/blogs/2007/11/30/volatile-almost-useless-for-multi-threaded-programming). 2007-11-30. URL: <https://software.intel.com/en-us/blogs/2007/11/30/volatile-almost-useless-for-multi-threaded-programming>

[Rust]

[Function std::ptr::read_volatile](https://doc.rust-lang.org/std/ptr/fn.read_volatile.html). URL: https://doc.rust-lang.org/std/ptr/fn.read_volatile.html

[SEBOR]

Martin Sebor. [C Defect Report #476: volatile semantics for lvalues](http://www.open-std.org/jtc1/sc22/wg14/www/docs/summary.htm#dr_476). 2015-08-26. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/summary.htm#dr_476

[SRST]

Patrick Rothfuss. [The Slow Regard of Silent Things](https://www.slowregard.com/2014/10/28/silent-things/). 2014-10-28.

[SysIII]

[SysIII/usr/src/stand/pdp11/iloader/console.c](https://minnie.tuhs.org/cgi-bin/utree.pl?file=SysIII/usr/src/stand/pdp11/iloader/console.c). URL: <https://minnie.tuhs.org/cgi-bin/utree.pl?file=SysIII/usr/src/stand/pdp11/iloader/console.c>

[TORVALDS]

Linus Torvalds. [GCC mailing list: Memory corruption due to word sharing](https://gcc.gnu.org/ml/gcc/2012-02/msg00027.html). 2012-02-01. URL: <https://gcc.gnu.org/ml/gcc/2012-02/msg00027.html>

[TROUBLE]

corbet. [The trouble with volatile](https://lwn.net/Articles/233479/). 2017-05-09. URL: <https://lwn.net/Articles/233479/>

[WHEN]

[When is a Volatile C++ Object Accessed?](https://gcc.gnu.org/onlinedocs/gcc/C_002b_002b-Volatiles.html). URL: https://gcc.gnu.org/onlinedocs/gcc/C_002b_002b-Volatiles.html

[WHY]

[Why does volatile exist?](https://stackoverflow.com/questions/72552/why-does-volatile-exist). 2008-07-16. URL: <https://stackoverflow.com/questions/72552/why-does-volatile-exist>

[WHYC]

[Why is volatile needed in C?](https://stackoverflow.com/questions/246127/why-is-volatile-needed-in-c). URL: <https://stackoverflow.com/questions/246127/why-is-volatile-needed-in-c>

[WMF]

Patrick Rothfuss. The Wise Man's Fear. 2011-03-01.

[XENXSA155]

Felix Wilhelm. [Xen XSA 155: Double fetches in paravirtualized devices](https://insinuator.net/2015/12/xen-xsa-155-double-fetches-in-paravirtualized-devices/). 2015-12-17. URL: <https://insinuator.net/2015/12/xen-xsa-155-double-fetches-in-paravirtualized-devices/>

[XTENSA]

[GCC Xtensa Options](https://gcc.gnu.org/onlinedocs/gcc-4.8.1/gcc/Xtensa-Options.html). URL: <https://gcc.gnu.org/onlinedocs/gcc-4.8.1/gcc/Xtensa-Options.html>

§ Issues Index

ISSUE 1 We would like guidance on `volatile` when combined with `operator->*`. That guidance will depend on choices made with respect to `volatile` aggregates. [↩](#)

ISSUE 2 There's a related problem in [\[TROUBLE\]](#) with chained assignments of `volatile` values, such as `a = b = c`. This is equally misleading, and it's not intuitive whether the value stored to `b` is re-read before storing to `a`. We would like guidance on whether this is worth addressing. [↩](#)

ISSUE 3 Which of the above approaches (deprecate `volatile`-qualified member functions, all-or-nothing, `struct volatile`, or deprecate `volatile` aggregates, should we pursue? [↩](#)

ISSUE 4 If we keep `volatile` aggregates, it seems like `volatile` aggregates should have constructors which initialize all members with `volatile` semantics, and destroy them with `volatile` semantics. Otherwise, we encourage the use of initialization / cleanup member functions. Alternatively, triviality could be mandated for constructors and destructors of `volatile` aggregates. The author is told by some embedded developers that it's very common to have an aggregate that describes some hardware, and to access hardware registers (member variables of the aggregate) by indirection through a pointer to `volatile` struct. [↩](#)

ISSUE 5 If we keep `volatile` aggregates, what does it mean to have a `volatile` virtual function table pointer? [↩](#)

ISSUE 6 It is unclear how `volatile union` should be accessed when all types in the `union` aren't stored using the same bits (i.e. should `union { char c; int i; }` always access the full union, even when only accessing `c`?). Some hardware defines different semantics to MMIO register accesses of different sizes to the same address. A union would be a natural way to represent such hardware. This could be outside the scope of the current paper. ↵

ISSUE 7 We would like guidance on whether `volatile` bit-fields should be constrained. At the moment no guarantee is made about read-modify-write operations required when mixing `volatile` and non-`volatile` bit-field data members. It seems like, at a minimum, bit fields should always cause alignment when subsequent data members change their cv-qualification. This could be outside the scope of the current paper. ↵

ISSUE 8 Should we go further and forbid `volatile` in containers? Some containers seem useful for signal handlers and such, however how loads and stores are performed to these containers isn't mandated by the Standard which means that to use containers in signal handlers one needs to synchronize separately with `atomic_signal_fence` and some form of token. Containers of `volatile` data are therefore misleading at best. ↵

ISSUE 9 We would like guidance on whether other read-modify-write operations should be maintained with implementation-defined semantics. Specifically, `exchange`, `compare_exchange_strong`, `compare_exchange_weak`, `fetch_add` / `fetch_sub` (for integral, pointer, and floating-point), and `fetch_and` / `fetch_or` / `fetch_xor` (for integral), can be given useful semantics by an implementation which wishes to guarantee that particular instructions will be emitted. This would maintain the status-quo whereby `volatile` is a semi-portable abstraction for hardware, and still allows us to consider deprecation in the future. Keeping these for now is the conservative option. ↵